

Álgebra Lineal Computacional con Python

Guía Teórica y Práctica con NumPy y SciPy

Curso de Programación y Álgebra Lineal

8 de septiembre de 2026

Resumen

El presente documento ofrece un compendio estructurado y detallado de las operaciones fundamentales del álgebra lineal aplicadas mediante el lenguaje de programación **Python**, utilizando principalmente las bibliotecas de cómputo científico **numpy** y **scipy.linalg**. Se cubren desde las estructuras básicas de vectores y matrices, operaciones aritméticas elementales, transformaciones, determinantes, cálculo de inversas y rango, hasta la resolución de sistemas de ecuaciones lineales, diagonalización (valores y vectores propios) y factorizaciones matriciales esenciales.

Índice

1. Introducción al Entorno de Cómputo Numérico	3
2. Vectores en $\mathbb{R}^n$	3
2.1. Creación de Vectores	3
2.2. Suma de Vectores y Multiplicación por Escalar	3
2.3. Producto Escalar (Producto Punto)	4
2.4. Norma y Distancia Euclidiana	4
2.5. Ángulo entre Vectores y Ortogonalidad	4
2.6. Producto Cruz (en $\mathbb{R}^3$)	5
3. Matrices en $\mathbb{R}^{m \times n}$	5
3.1. Definición de Matrices Especiales	5
3.2. Multiplicación Matricial	5
3.3. Transpuesta y Traza	6
4. Determinantes, Inversas y Rango	6
4.1. Determinante	6
4.2. Matriz Inversa	6
4.3. Rango de una Matriz	7
5. Sistemas de Ecuaciones Lineales	7
5.1. Sistemas Cuadrados Determinados ($m = n$)	7
5.2. Sistemas Sobredeterminados (Mínimos Cuadrados)	8
6. Valores y Vectores Propios (Eigenvalues y Eigenvectors)	8
7. Descomposiciones Matriciales Fundamentales	8
7.1. Descomposición LU	8
7.2. Descomposición en Valores Singulares (SVD)	9
8. Tabla Resumen de Equivalencias Matemáticas y Código	9

1. Introducción al Entorno de Cómputo Numérico

En la computación científica, el álgebra lineal es la columna vertebral de disciplinas como el aprendizaje automático (*Machine Learning*), el procesamiento de imágenes, gráficos por computadora y la simulación física. En Python, la biblioteca estándar para el manejo de tensores y álgebra lineal es **NumPy** (*Numerical Python*), complementada por el submódulo `scipy.linalg` de **SciPy**.

Para iniciar cualquier proyecto de álgebra lineal en Python, se importan los siguientes módulos:

```
1 import numpy as np
2 import scipy.linalg as la
```

Ventajas de NumPy frente a listas nativas

A diferencia de las listas tradicionales de Python, los arreglos de NumPy (`np.ndarray`):

- Almacenan datos en bloques contiguos de memoria con tipos homogéneos.
- Permiten la **vectorización** de operaciones, ejecutadas en C o Fortran de forma paralela y optimizada.
- Se integran directamente con librerías de alto rendimiento como BLAS y LAPACK.

2. Vectores en $\mathbb{R}^n$

Un vector $\mathbf{v} \in \mathbb{R}^n$ se representa como una tupla ordenada de n componentes reales:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} \quad \text{o bien} \quad \mathbf{v}^T = [v_1, v_2, \dots, v_n] \quad (1)$$

2.1. Creación de Vectores

En NumPy, un vector se construye comúnmente como un arreglo unidimensional:

```
1 # Creacion de vectores en R^3
2 u = np.array([1.0, 2.0, 3.0])
3 v = np.array([4.0, 0.0, -1.0])
4
5 print("Vector u:", u)
6 print("Dimension (shape):", u.shape)
```

2.2. Suma de Vectores y Multiplicación por Escalar

Sean $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ y $c \in \mathbb{R}$. Las operaciones elemento a elemento se definen como:

$$\mathbf{u} + \mathbf{v} = [u_1 + v_1, u_2 + v_2, \dots, u_n + v_n]^T \quad (2)$$

$$c\mathbf{u} = [cu_1, cu_2, \dots, cu_n]^T \quad (3)$$

```

1 # Suma y resta vectorial
2 suma = u + v          # [5., 2., 2.]
3 resta = u - v         # [-3., 2., 4.]
4
5 # Multiplicacion por escalar
6 c = 2.5
7 escalar_u = c * u     # [2.5, 5.0, 7.5]

```

2.3. Producto Escalar (Producto Punto)

El producto punto entre $\mathbf{u}$ y $\mathbf{v}$ produce un escalar:

$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^n u_i v_i = \mathbf{u}^T \mathbf{v} \quad (4)$$

En Python se puede computar usando el operador infijo `@` o con `np.dot()`:

```

1 # Producto punto
2 prod_punto_1 = np.dot(u, v)
3 prod_punto_2 = u @ v          # Operador preferido en Python moderno
4
5 print("u . v =", prod_punto_2) # 1*4 + 2*0 + 3*(-1) = 1.0

```

2.4. Norma y Distancia Euclidiana

La norma L_2 (o longitud euclidiana) de un vector se define como:

$$\|\mathbf{v}\|_2 = \sqrt{\mathbf{v} \cdot \mathbf{v}} = \sqrt{\sum_{i=1}^n v_i^2} \quad (5)$$

```

1 # Modulo / Norma del vector
2 norma_u = np.linalg.norm(u)
3 print("||u||_2 =", norma_u)
4
5 # Vector unitario (normalizacion)
6 u_unitario = u / np.linalg.norm(u)
7
8 # Distancia euclidiana entre u y v
9 distancia = np.linalg.norm(u - v)
10 print("d(u, v) =", distancia)

```

2.5. Ángulo entre Vectores y Ortogonalidad

A partir de la relación $\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \|\mathbf{v}\| \cos(\theta)$, el ángulo θ es:

$$\theta = \arccos \left(\frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} \right) \quad (6)$$

```

1 cos_theta = np.dot(u, v) / (np.linalg.norm(u) * np.linalg.norm(v))
2 # Aseguramos el rango [-1, 1] para evitar inconsistencias numericas
3 cos_theta = np.clip(cos_theta, -1.0, 1.0)
4 angulo_rad = np.arccos(cos_theta)
5 angulo_deg = np.degrees(angulo_rad)
6 print(f"Angulo: {angulo_rad:.4f} rad ({angulo_deg:.2f} deg)")

```

Dos vectores son **ortogonales** si y sólo si $\mathbf{u} \cdot \mathbf{v} = 0$.

2.6. Producto Cruz (en $\mathbb{R}^3$)

Para vectores tridimensionales, el producto vectorial produce un vector ortogonal a ambos:

$$\mathbf{u} \times \mathbf{v} = \det \begin{bmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ u_1 & u_2 & u_3 \\ v_1 & v_2 & v_3 \end{bmatrix} \quad (7)$$

```
1 prod_cruz = np.cross(u, v)
2 print("u x v =", prod_cruz)
```

3. Matrices en $\mathbb{R}^{m \times n}$

Una matriz $A \in \mathbb{R}^{m \times n}$ consta de m filas y n columnas:

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \quad (8)$$

3.1. Definición de Matrices Especiales

```
1 # Matriz general explicita de 2x3
2 A = np.array([
3     [1.0, 2.0, 3.0],
4     [4.0, 5.0, 6.0]
5 ])
6
7 # Matrices constantes
8 I3 = np.eye(3)           # Matriz identidad 3x3
9 Z = np.zeros((2, 3))    # Matriz de ceros 2x3
10 O = np.ones((3, 3))     # Matriz de unos 3x3
11 D = np.diag([2, 5, -1]) # Matriz diagonal con elementos dados
```

3.2. Multiplicación Matricial

Sean $A \in \mathbb{R}^{m \times k}$ y $B \in \mathbb{R}^{k \times n}$. El producto $C = AB \in \mathbb{R}^{m \times n}$ está dado por:

$$c_{ij} = \sum_{l=1}^k a_{il}b_{lj} \quad (9)$$

¡Cuidado con la multiplicación elemento a elemento!

En NumPy:

- $A * B$ calcula el **producto de Hadamard** (elemento a elemento). Requiere que A y B tengan dimensiones idénticas o compatibles por difusión (*broadcasting*).
- $A @ B$ o `np.matmul(A, B)` ejecuta el **producto matricial estándar**.

```

1 B = np.array([
2     [7.0, 8.0],
3     [9.0, 1.0],
4     [2.0, 3.0]
5 ]) # Dimension 3x2
6
7 # Multiplicacion matricial: (2x3) x (3x2) -> (2x2)
8 C = A @ B
9 print("Producto matricial A @ B:\n", C)

```

3.3. Transpuesta y Traza

La transpuesta A^T intercambia filas por columnas ($a_{ij}^T = a_{ji}$). Para una matriz cuadrada $M \in \mathbb{R}^{n \times n}$, la traza es la suma de sus elementos diagonales:

$$\text{tr}(M) = \sum_{i=1}^n m_{ii} \quad (10)$$

```

1 A_transpuesta = A.T
2 print("Transpuesta de A (3x2):\n", A_transpuesta)
3
4 # Traza de una matriz cuadrada
5 M = np.array([[3, 1], [-2, 4]])
6 traza_M = np.trace(M)
7 print("Traza de M:", traza_M) # 3 + 4 = 7

```

4. Determinantes, Inversas y Rango

4.1. Determinante

El determinante $\det(A)$ es una propiedad escalar exclusiva de matrices cuadradas. Describe cómo la transformación lineal escala el hipervolumen de los objetos.

```

1 A_cuad = np.array([
2     [2.0, 1.0, 3.0],
3     [1.0, 0.0, 2.0],
4     [4.0, 1.0, -1.0]
5 ])
6
7 det_A = np.linalg.det(A_cuad)
8 print(f"det(A) = {det_A:.4f}")

```

4.2. Matriz Inversa

Una matriz $A \in \mathbb{R}^{n \times n}$ es invertible o no singular si existe A^{-1} tal que:

$$AA^{-1} = A^{-1}A = I_n \quad (11)$$

Esto es posible si y sólo si $\det(A) \neq 0$.

```

1 if not np.isclose(det_A, 0.0):
2     A_inv = np.linalg.inv(A_cuad)
3     print("Inversa de A:\n", A_inv)
4     # Verificacion: A @ A_inv ~ I

```

```

5     print("Verificacion I:\n", np.round(A_cuad @ A_inv, 6))
6 else:
7     print("La matriz es singular y no posee inversa.")

```

4.3. Rango de una Matriz

El rango indica el número de columnas (o filas) linealmente independientes:

```

1 rango = np.linalg.matrix_rank(A_cuad)
2 print("Rango de la matriz:", rango)

```

5. Sistemas de Ecuaciones Lineales

Un sistema de m ecuaciones lineales con n incógnitas se escribe en forma matricial como:

$$A\mathbf{x} = \mathbf{b} \quad (12)$$

donde $A \in \mathbb{R}^{m \times n}$ es la matriz de coeficientes, $\mathbf{x} \in \mathbb{R}^n$ es el vector incógnita y $\mathbf{b} \in \mathbb{R}^m$ es el vector de términos independientes.

5.1. Sistemas Cuadrados Determinados ($m = n$)

Para un sistema con solución única, se utiliza `np.linalg.solve()`:

¿Por qué evitar la inversión explícita de matrices en código?

Calcular la inversa explícita A^{-1} es computacionalmente ineficiente ($O(n^3)$ con constante alta) e introduce mayor error numérico por pérdida de precisión flotante. La función `np.linalg.solve(A, b)` utiliza internamente la descomposición LU con pivoteo parcial, siendo mucho más rápida y estable.

```

1 # Resolver:
2 # 2x + y + 3z = 1
3 #   x      + 2z = 2
4 # 4x + y -  z = 3
5
6 A_sys = np.array([
7     [2.0, 1.0, 3.0],
8     [1.0, 0.0, 2.0],
9     [4.0, 1.0, -1.0]
10 ])
11 b_sys = np.array([1.0, 2.0, 3.0])
12
13 # Solucion directa y estable
14 x = np.linalg.solve(A_sys, b_sys)
15 print("Solucion x:", x)
16
17 # Verificacion del residuo ||Ax - b||
18 residuo = np.linalg.norm(A_sys @ x - b_sys)
19 print(f"Norma del residuo: {residuo:.2e}")

```

5.2. Sistemas Sobredeterminados (Mínimos Cuadrados)

Cuando hay más ecuaciones que incógnitas ($m > n$), comúnmente no existe solución exacta. Se busca minimizar el residuo $\|Ax - b\|_2^2$:

```
1 # 4 observaciones, 2 incógnitas
2 A_over = np.array([[1.0, 1.0], [1.0, 2.0], [1.0, 3.0], [1.0, 4.0]])
3 b_over = np.array([2.1, 2.9, 4.2, 4.9])
4
5 # Ajuste por mínimos cuadrados (lstsq)
6 sol_lstsq, residuos, rank, s = np.linalg.lstsq(A_over, b_over, rcond=
7     None)
8 print("Parametros estimados (pendiente, intercepto):", sol_lstsq)
```

6. Valores y Vectores Propios (Eigenvalues y Eigenvectors)

Para una matriz cuadrada $A \in \mathbb{R}^{n \times n}$, un escalar λ y un vector no nulo $v \neq 0$ son respectivamente un valor propio y un vector propio si:

$$Av = \lambda v \iff (A - \lambda I)v = 0 \quad (13)$$

```
1 M_eigen = np.array([
2     [4.0, 1.0],
3     [2.0, 3.0]
4 ])
5
6 # Calculo de autovalores y autovectores
7 valores, vectores = np.linalg.eig(M_eigen)
8
9 print("Valores propios (lambda):", valores)
10 # Nota: Las columnas de 'vectores' son los vectores propios
    correspondientes
11 for i in range(len(valores)):
12     lam = valores[i]
13     v_i = vectores[:, i]
14     print(f"Autovalor {i+1}: {lam:.4f}")
15     print(f"Autovector {i+1}: {v_i}")
16     # Comprobacion: A v = lambda v
17     print("Diferencia ||Av - lambda v||:",
18         np.linalg.norm(M_eigen @ v_i - lam * v_i))
```

Matrices Simétricas y np.linalg.eigh

Si la matriz es simétrica ($A = A^T$), todos sus valores propios son reales y sus vectores propios son mutuamente ortonormales. Para estos casos, se recomienda utilizar `np.linalg.eigh`, que aprovecha algoritmos especializados mucho más rápidos y numéricamente estables.

7. Descomposiciones Matriciales Fundamentales

7.1. Descomposición LU

Factoriza una matriz A en el producto de una matriz de permutación P , una triangular inferior L y una triangular superior U :

$$PA = LU \quad (14)$$

```

1 # Utilizando scipy.linalg.lu
2 P, L, U = la.lu(A_cuad)
3
4 print("Matriz L (Triangular inferior):\n", np.round(L, 3))
5 print("Matriz U (Triangular superior):\n", np.round(U, 3))

```

7.2. Descomposición en Valores Singulares (SVD)

Cualquier matriz $A \in \mathbb{R}^{m \times n}$ puede factorizarse en:

$$A = U\Sigma V^T \quad (15)$$

donde $U \in \mathbb{R}^{m \times m}$ y $V \in \mathbb{R}^{n \times n}$ son ortogonales, y $\Sigma \in \mathbb{R}^{m \times n}$ contiene los valores singulares ordenados en su diagonal.

```

1 U_svd, s_svd, Vt_svd = np.linalg.svd(A)
2
3 print("Valores singulares sigma:", s_svd)
4 print("Forma de U:", U_svd.shape)
5 print("Forma de V^T:", Vt_svd.shape)

```

8. Tabla Resumen de Equivalencias Matemáticas y Código

Operación Matemática	Simbología	Sintaxis en Python (NumPy)
Suma de vectores/matrices	$A + B$	<code>A + B</code>
Multiplicación por escalar	αA	<code>alpha * A</code>
Producto escalar	$\mathbf{u} \cdot \mathbf{v}$	<code>u @ v</code> o <code>np.dot(u, v)</code>
Producto vectorial (3D)	$\mathbf{u} \times \mathbf{v}$	<code>np.cross(u, v)</code>
Norma euclidiana (L_2)	$\ \mathbf{x}\ _2$	<code>np.linalg.norm(x)</code>
Multiplicación de matrices	AB	<code>A @ B</code> o <code>np.matmul(A, B)</code>
Producto elemento a elemento	$A \odot B$	<code>A * B</code>
Transpuesta	A^T	<code>A.T</code>
Traza	$\text{tr}(A)$	<code>np.trace(A)</code>
Determinante	$\det(A)$	<code>np.linalg.det(A)</code>
Inversa	A^{-1}	<code>np.linalg.inv(A)</code>
Rango matricial	$\text{rang}(A)$	<code>np.linalg.matrix_rank(A)</code>
Resolución de $A\mathbf{x} = \mathbf{b}$	$\mathbf{x} = A^{-1}\mathbf{b}$	<code>np.linalg.solve(A, b)</code>
Valores y vectores propios	$A\mathbf{v} = \lambda\mathbf{v}$	<code>np.linalg.eig(A)</code>
Valores/vectores propios (simétrica)	$A\mathbf{v} = \lambda\mathbf{v}$	<code>np.linalg.eigh(A)</code>
Descomposición SVD	$A = U\Sigma V^T$	<code>np.linalg.svd(A)</code>

Tabla 1: Referencia rápida de operaciones de álgebra lineal en Python.

9. Conclusión

Python, a través de NumPy y SciPy, proporciona un marco computacional robusto, expresivo y de alto rendimiento para el álgebra lineal. Comprender la correspondencia entre la notación matemática formal y su equivalente computacional es esencial para implementar modelos numéricos eficientes, estables y libres de cuellos de botella.

```
import numpy as np
arreglo = np.array ([10 , 20 , 30 , 40 , 50])
suma = 0

for valor in arreglo:
    suma += valor

print (f"La suma del arreglo es: {suma}")
```

La suma del arreglo es: 150

```
matriz = [
    [1 , 2 , 3],
    [4 , 5 , 6],
    [7 , 8 , 9]
]

for fila in matriz:
    for elemento in fila:
        print(elemento, end = " ")
    print() # Salto de línea al terminar cada fila
```

1 2 3
4 5 6
7 8 9

```
for fila in matriz:
    print(fila)
```

[1, 2, 3]
[4, 5, 6]
[7, 8, 9]

```
for fila in matriz:
    for elemento in fila:
        print(elemento, end = " ")
    print()
```

1 2 3
4 5 6
7 8 9

```
MS = [
    [3 , 5 , 4],
    [-1 , 2 , 7],
]

print(MS)
```

[[3, 5, 4], [-1, 2, 7]]

```
for fila in MS:
    for elemento in fila:
        print(elemento, end = " ")
    print()
```

3 5 4
-1 2 7

```
print(MS[0][1])
```

5

```
def sofia_det2x2(A):
    a=A[0][0]
    b=A[0][1]
    c=A[1][0]
    d=A[1][1]
    y=a*d-b*c
    return y
```

```
A3= [
    [1, 3],
    [4, 5],
]
print(A3)
sofia_det2x2(A3)
```

```
[[1, 3], [4, 5]]
-7
```

```
def sofia_det3x3(B):
    a=B[0][0]
    b=B[0][1]
    c=B[0][2]
    d=B[1][0]
    e=B[1][1]
    f=B[1][2]
    g=B[2][0]
    h=B[2][1]
    i=B[2][2]
    B1=[
        [e, f],
        [h, i],
    ]

    B2=[
        [d, f],
        [g, i],
    ]

    B3=[
        [d, e],
        [g, h],
    ]
    y=a*sofia_det2x2(B1)-b*sofia_det2x2(B2)+c*sofia_det2x2(B3)
    return y
```

```
Z = [
    [1, 2, 7],
    [4, 3, 4],
    [5, 6, 7]
]
sofia_det3x3(Z)
```

44

```
x=sofia_det2x2([[4,2],[8,5]])/sofia_det2x2([[3,2],[-1,5]])
print(f"El resultado de la x es {x:.3f}")
```

El resultado de la x es 0.235

```
y=sofia_det2x2([[3,4],[-1,8]])/sofia_det2x2([[3,2],[-1,5]])
print(f"El resultado de la y es {y:.3f}")
```

El resultado de la y es 1.647


```
import numpy as np
A=np.array([[1, -1, 4],[2, 3, 12]])
#print(A)
B=np.array([[2, -1, 3],[1, 0, 4]])
#print(B)
C=np.array([[1, 4],[-5, 3]])
#print(C)
#print(A+B)
escalar=-1
print(f"La matriz A es igual a: {A}, y el producto por escalar es igual a: {escalar*A}")
```

```
La matriz A es igual a: [[ 1 -1  4]
 [ 2  3 12]], y el producto por escalar es igual a: [[ -1  1 -4]
 [ -2 -3 -12]]
```

```
D=np.matrix([[3, 1, 0],[0, 1, 1]])
print(B+D)
```

```
[[5 0 3]
 [1 1 5]]
```

```
A1=np.matrix([[1, 3, 4],[5, 1, 1]])
B1=np.matrix([[1, 3],[1, 4]])
C1=np.matrix([[1, 3],[1, 4],[-1, 5]])
```

```
print(B1*A1)
```

```
print(A1*C1)
```

```
[[16  6  7]
 [21  7  8]]
[[ 0 35]
 [ 5 24]]
```

```
import numpy as np
from numpy import matrix

# Definición de las matrices con el comando matrix
A = matrix([[1, 2],[3, 4],[5, 6]])      # 3x2

B = matrix([[1, 1],[2, 2],[3, 3],[4, 4],[5, 5]])      # 5x2

C = matrix([[7, 8],[9, 10],[11, 12]])      # 3x2

D = matrix([[1, 2],[3, 4]])      # 2x2

# 1) SUMA (solo A + C porque comparten dimensiones 3x2)

suma_AC = A + C
print("A + C =\n", suma_AC)
# B no se puede sumar con ninguna porque es 5x2

# 2) RESTA (solo entre A y C)

resta_AC = A - C
resta_CA = C - A
print(f"A - C = {resta_AC}")
print(f"C - A = {resta_CA}")

# 3) MULTIPLICACIÓN POR ESCALAR (escalar = 3) -> todas se pueden
print("MULTIPLICACIÓN POR ESCALAR (3)")
print(3 * A)
print(3 * B)
print(3 * C)
print(3 * D)

# 4) MULTIPLICACIÓN ENTRE MATRICES
# A(3x2) · D(2x2) = 3x2
# C(3x2) · D(2x2) = 3x2
# B(5x2) no es compatible con ninguna (necesitaría una 2x5)

mult_AD = A * D
mult_CD = C * D
print(f"A * D = {mult_AD}")
```

```
print(f"C * D = {mult_CD}")
```

```
A + C =
```

```
[[ 8 10]
```

```
[12 14]
```

```
[16 18]]
```

```
A - C = [[-6 -6]
```

```
[-6 -6]
```

```
[-6 -6]]
```

```
C - A = [[6 6]
```

```
[6 6]
```

```
[6 6]]
```

```
MULTIPLICACIÓN POR ESCALAR (3)
```

```
[[ 3 6]
```

```
[ 9 12]
```

```
[15 18]]
```

```
[[ 3 3]
```

```
[ 6 6]
```

```
[ 9 9]
```

```
[12 12]
```

```
[15 15]]
```

```
[[21 24]
```

```
[27 30]
```

```
[33 36]]
```

```
[[ 3 6]
```

```
[ 9 12]]
```

```
A * D = [[ 7 10]
```

```
[15 22]
```

```
[23 34]]
```

```
C * D = [[31 46]
```

```
[39 58]
```

```
[47 70]]
```

Operaciones entre Matrices con NumPy en Python

Documento de Referencia

September 8, 2026

Contents

1	Introducción	3
2	Instalación e Importación	3
3	Creación de Matrices	3
4	Operaciones Aritméticas Básicas	4
4.1	Suma de Matrices	4
4.2	Resta de Matrices	4
4.3	Multiplicación Escalar	5
4.4	División Escalar	5
5	Multiplicación de Matrices	5
5.1	Multiplicación Elemento a Elemento (Producto de Hadamard)	5
5.2	Multiplicación de Matrices (Producto Matricial)	6
6	Transposición de Matrices	7
7	Potencia de Matrices	7
8	Determinante de una Matriz	8
9	Matriz Inversa	8
10	Rango de una Matriz	9
11	Trazas de Matrices	9
12	Normas de Matrices	10
13	Operaciones con Vectores	11
13.1	Producto Vectorial (Producto Cruz)	11
13.2	Producto Punto (Producto Escalar)	11
14	Ejemplo Completo	11

1 Introducción

NumPy es una biblioteca fundamental para el cálculo científico en Python. Proporciona un objeto de matriz multidimensional de alto rendimiento y herramientas para trabajar con estos arreglos. En este documento, exploraremos las operaciones básicas entre matrices utilizando NumPy.

2 Instalación e Importación

Para comenzar a trabajar con NumPy, primero debemos instalarlo:

```
1 pip install numpy
```

Listing 1: Instalación de NumPy

Luego, lo importamos en nuestro script:

```
1 import numpy as np
```

Listing 2: Importación de NumPy

3 Creación de Matrices

Primero, veamos cómo crear matrices en NumPy:

```
1 import numpy as np
2
3 # Crear una matriz 2x3
4 A = np.array([[1, 2, 3],
5               [4, 5, 6]])
6
7 # Crear otra matriz 3x2 para multiplicación
8 B = np.array([[7, 8],
9               [9, 10],
10              [11, 12]])
11
12 # Crear una matriz identidad 3x3
13 I = np.eye(3)
14
15 # Crear una matriz de ceros
16 Z = np.zeros((2, 3))
17
18 # Crear una matriz de unos
19 O = np.ones((2, 3))
20
21 # Crear una matriz de valores aleatorios
22 R = np.random.rand(2, 3)
23
24 print("Matriz A:")
25 print(A)
26 print("\nMatriz B:")
27 print(B)
```

4 Operaciones Aritméticas Básicas

4.1 Suma de Matrices

La suma de matrices se realiza elemento a elemento. Las matrices deben tener la misma dimensión.

```
1 import numpy as np
2
3 A = np.array([[1, 2], [3, 4]])
4 B = np.array([[5, 6], [7, 8]])
5
6 # Suma de matrices
7 C = A + B
8 print("Suma de matrices (A + B):")
9 print(C)
10
11 # Tambi n se puede usar np.add()
12 D = np.add(A, B)
13 print("\nSuma usando np.add():")
14 print(D)
15
16 # Ejemplo con matrices de diferente tama o (esto genera error)
17 # E = np.array([[1, 2, 3], [4, 5, 6]])
18 # F = A + E # Error: dimensiones incompatibles
```

Listing 4: Suma de Matrices

4.2 Resta de Matrices

La resta también se realiza elemento a elemento.

```
1 import numpy as np
2
3 A = np.array([[5, 6], [7, 8]])
4 B = np.array([[1, 2], [3, 4]])
5
6 # Resta de matrices
7 C = A - B
8 print("Resta de matrices (A - B):")
9 print(C)
10
11 # Tambi n se puede usar np.subtract()
12 D = np.subtract(A, B)
13 print("\nResta usando np.subtract():")
14 print(D)
```

Listing 5: Resta de Matrices

4.3 Multiplicación Escalar

Multiplicar una matriz por un escalar (número).

```
1 import numpy as np
2
3 A = np.array([[1, 2], [3, 4]])
4 k = 3
5
6 # Multiplicación por escalar
7 B = k * A
8 print(f"Multiplicación de A por {k}:")
9 print(B)
10
11 # También se puede usar np.multiply()
12 C = np.multiply(k, A)
13 print(f"\nMultiplicación usando np.multiply():")
14 print(C)
```

Listing 6: Multiplicación Escalar

4.4 División Escalar

Dividir una matriz por un escalar.

```
1 import numpy as np
2
3 A = np.array([[6, 8], [10, 12]])
4 k = 2
5
6 # División por escalar
7 B = A / k
8 print(f"División de A entre {k}:")
9 print(B)
10
11 # También se puede usar np.divide()
12 C = np.divide(A, k)
13 print(f"\nDivisión usando np.divide():")
14 print(C)
```

Listing 7: División Escalar

5 Multiplicación de Matrices

5.1 Multiplicación Elemento a Elemento (Producto de Hadamard)

Esta operación multiplica los elementos correspondientes de dos matrices del mismo tamaño.

```
1 import numpy as np
2
3 A = np.array([[1, 2], [3, 4]])
```

```

4 B = np.array([[5, 6], [7, 8]])
5
6 # Multiplicaci n elemento a elemento
7 C = A * B
8 print("Multiplicaci n elemento a elemento (A * B):")
9 print(C)
10
11 # Tambi n se puede usar np.multiply()
12 D = np.multiply(A, B)
13 print("\nMultiplicaci n usando np.multiply():")
14 print(D)

```

Listing 8: Multiplicaci3n Elemento a Elemento

5.2 Multiplicaci3n de Matrices (Producto Matricial)

La multiplicaci3n de matrices sigue las reglas del 3lgebra lineal. El n3mero de columnas de la primera matriz debe ser igual al n3mero de filas de la segunda.

```

1 import numpy as np
2
3 A = np.array([[1, 2, 3],
4               [4, 5, 6]])
5
6 B = np.array([[7, 8],
7               [9, 10],
8               [11, 12]])
9
10 # Multiplicaci n matricial (2x3 * 3x2 = 2x2)
11 C = np.dot(A, B)
12 print("Multiplicaci n matricial (np.dot):")
13 print(C)
14
15 # Tambi n se puede usar el operador @
16 D = A @ B
17 print("\nMultiplicaci n matricial usando @:")
18 print(D)
19
20 # O usar np.matmul()
21 E = np.matmul(A, B)
22 print("\nMultiplicaci n matricial usando np.matmul():")
23 print(E)
24
25 # Ejemplo con matrices cuadradas
26 F = np.array([[1, 2], [3, 4]])
27 G = np.array([[5, 6], [7, 8]])
28 H = F @ G
29 print("\nMultiplicaci n de matrices cuadradas:")
30 print(H)

```

Listing 9: Multiplicaci3n de Matrices

6 Transposición de Matrices

La transposición de una matriz intercambia sus filas por columnas.

```
1 import numpy as np
2
3 A = np.array([[1, 2, 3],
4               [4, 5, 6]])
5
6 # Transposición
7 B = A.T
8 print("Matriz original A:")
9 print(A)
10 print("\nMatriz transpuesta A.T:")
11 print(B)
12
13 # Para matrices complejas, .T no conjuga los elementos
14 C = np.array([[1+2j, 3+4j], [5+6j, 7+8j]])
15 D = C.T # Solo transpone, no conjuga
16 print("\nMatriz compleja C:")
17 print(C)
18 print("\nTranspuesta C.T:")
19 print(D)
20
21 # Para obtener la transpuesta conjugada (Hermitiana)
22 E = C.conj().T
23 print("\nTranspuesta conjugada (Hermitiana):")
24 print(E)
```

Listing 10: Transposición de Matrices

7 Potencia de Matrices

Elevar una matriz a una potencia (solo para matrices cuadradas).

```
1 import numpy as np
2
3 A = np.array([[1, 2],
4               [3, 4]])
5
6 # Potencia de matriz (A^2 = A * A)
7 B = np.linalg.matrix_power(A, 2)
8 print("Matriz A:")
9 print(A)
10 print("\nA^2:")
11 print(B)
12
13 # A^3
14 C = np.linalg.matrix_power(A, 3)
15 print("\nA^3:")
16 print(C)
17
```

```

18 # A^0 = Matriz identidad
19 D = np.linalg.matrix_power(A, 0)
20 print("\nA^0 (Identidad):")
21 print(D)

```

Listing 11: Potencia de Matrices

8 Determinante de una Matriz

El determinante solo se define para matrices cuadradas.

```

1 import numpy as np
2
3 A = np.array([[1, 2],
4               [3, 4]])
5
6 # Calcular el determinante
7 det_A = np.linalg.det(A)
8 print(f"Matriz A:")
9 print(A)
10 print(f"\nDeterminante de A: {det_A:.2f}")
11
12 # Ejemplo con matriz 3x3
13 B = np.array([[1, 2, 3],
14               [4, 5, 6],
15               [7, 8, 9]])
16 det_B = np.linalg.det(B)
17 print(f"\nMatriz B:")
18 print(B)
19 print(f"\nDeterminante de B: {det_B:.2f}")

```

Listing 12: Determinante de Matrices

9 Matriz Inversa

La matriz inversa solo existe para matrices cuadradas con determinante no nulo.

```

1 import numpy as np
2
3 A = np.array([[4, 7],
4               [2, 6]])
5
6 # Calcular la inversa
7 A_inv = np.linalg.inv(A)
8 print("Matriz A:")
9 print(A)
10 print("\nMatriz inversa de A:")
11 print(A_inv)
12
13 # Verificar que A * A_inv = I

```

```

14 I = A @ A_inv
15 print("\nA * A_inv (deber a ser la identidad):")
16 print(I.round(2))
17
18 # Ejemplo con matriz singular (sin inversa)
19 B = np.array([[1, 2],
20               [2, 4]])
21 print("\nMatriz B (singular):")
22 print(B)
23 try:
24     B_inv = np.linalg.inv(B)
25 except np.linalg.LinAlgError:
26     print("Error: La matriz B es singular y no tiene inversa")

```

Listing 13: Matriz Inversa

10 Rango de una Matriz

El rango de una matriz es el número de filas o columnas linealmente independientes.

```

1 import numpy as np
2
3 A = np.array([[1, 2, 3],
4               [4, 5, 6],
5               [7, 8, 9]])
6
7 # Calcular el rango
8 rango = np.linalg.matrix_rank(A)
9 print("Matriz A:")
10 print(A)
11 print(f"\nRango de A: {rango}")
12
13 B = np.array([[1, 0, 0],
14               [0, 1, 0],
15               [0, 0, 1]])
16 print("\nMatriz B (identidad):")
17 print(B)
18 print(f"\nRango de B: {np.linalg.matrix_rank(B)}")
19
20 C = np.array([[1, 2],
21               [2, 4]])
22 print("\nMatriz C (singular):")
23 print(C)
24 print(f"\nRango de C: {np.linalg.matrix_rank(C)}")

```

Listing 14: Rango de Matrices

11 Trazas de Matrices

La traza de una matriz es la suma de los elementos de la diagonal principal.

```

1 import numpy as np
2
3 A = np.array([[1, 2, 3],
4               [4, 5, 6],
5               [7, 8, 9]])
6
7 # Calcular la traza
8 traza = np.trace(A)
9 print("Matriz A:")
10 print(A)
11 print(f"\nTraza de A: {traza}")
12
13 # Tambi n se puede calcular sumando la diagonal
14 traza_directa = np.diag(A).sum()
15 print(f"Traza sumando la diagonal: {traza_directa}")
16
17 # Propiedad: traza(A) = traza(A.T)
18 print(f"Traza(A.T): {np.trace(A.T)}")

```

Listing 15: Traza de Matrices

12 Normas de Matrices

Las normas son funciones que asignan una longitud o tamaño a una matriz.

```

1 import numpy as np
2
3 A = np.array([[1, 2],
4               [3, 4]])
5
6 # Norma de Frobenius
7 norm_fro = np.linalg.norm(A, 'fro')
8 print("Matriz A:")
9 print(A)
10 print(f"\nNorma de Frobenius: {norm_fro:.2f}")
11
12 # Norma 1 (m x ima suma de columnas)
13 norm_1 = np.linalg.norm(A, 1)
14 print(f"Norma 1: {norm_1:.2f}")
15
16 # Norma infinita (m x ima suma de filas)
17 norm_inf = np.linalg.norm(A, np.inf)
18 print(f"Norma infinita: {norm_inf:.2f}")
19
20 # Norma 2 (espectral)
21 norm_2 = np.linalg.norm(A, 2)
22 print(f"Norma 2 (espectral): {norm_2:.2f}")

```

Listing 16: Normas de Matrices

13 Operaciones con Vectores

13.1 Producto Vectorial (Producto Cruz)

```
1 import numpy as np
2
3 v1 = np.array([1, 2, 3])
4 v2 = np.array([4, 5, 6])
5
6 # Producto cruz
7 v3 = np.cross(v1, v2)
8 print(f"v1 = {v1}")
9 print(f"v2 = {v2}")
10 print(f"v1      v2 = {v3}")
11
12 # Verificar que v3 es ortogonal a v1 y v2
13 print(f"v1      (v1      v2) = {np.dot(v1, v3)}")
14 print(f"v2      (v1      v2) = {np.dot(v2, v3)}")
```

Listing 17: Producto Vectorial

13.2 Producto Punto (Producto Escalar)

```
1 import numpy as np
2
3 v1 = np.array([1, 2, 3])
4 v2 = np.array([4, 5, 6])
5
6 # Producto punto
7 producto_escalar = np.dot(v1, v2)
8 print(f"v1 = {v1}")
9 print(f"v2 = {v2}")
10 print(f"v1      v2 = {producto_escalar}")
11
12 # Tambi n se puede usar v1 @ v2
13 print(f"v1 @ v2 = {v1 @ v2}")
14
15 # O np.inner
16 print(f"np.inner(v1, v2) = {np.inner(v1, v2)}")
```

Listing 18: Producto Punto

14 Ejemplo Completo

A continuación, un ejemplo completo que combina varias operaciones:

```
1 import numpy as np
2
3 # Creaci n de matrices
4 A = np.array([[1, 2, 3],
```

```

5         [4, 5, 6],
6         [7, 8, 9]])
7
8 B = np.array([[9, 8, 7],
9               [6, 5, 4],
10              [3, 2, 1]])
11
12 print("=" * 50)
13 print("OPERACIONES CON MATRICES")
14 print("=" * 50)
15
16 print("\nMatriz A:")
17 print(A)
18 print("\nMatriz B:")
19 print(B)
20
21 # Suma
22 print("\n" + "-" * 30)
23 print("SUMA (A + B):")
24 print(A + B)
25
26 # Resta
27 print("\n" + "-" * 30)
28 print("RESTA (A - B):")
29 print(A - B)
30
31 # Multiplicaci n escalar
32 k = 2
33 print(f"\n" + "-" * 30)
34 print(f"MULTIPLICACI N POR ESCALAR ({k} * A):")
35 print(k * A)
36
37 # Multiplicaci n elemento a elemento
38 print("\n" + "-" * 30)
39 print("MULTIPLICACI N ELEMENTO A ELEMENTO (A * B):")
40 print(A * B)
41
42 # Multiplicaci n matricial
43 print("\n" + "-" * 30)
44 print("MULTIPLICACI N MATRICIAL (A @ B):")
45 print(A @ B)
46
47 # Transposici n
48 print("\n" + "-" * 30)
49 print("TRANSPOSICI N (A.T):")
50 print(A.T)
51
52 # Determinante
53 print("\n" + "-" * 30)
54 print("DETERMINANTES:")
55 print(f"det(A) = {np.linalg.det(A):.2f}")

```

```

56 print(f"det(B) = {np.linalg.det(B):.2f}")
57
58 # Inversa (si existe)
59 print("\n" + "-" * 30)
60 print("INVERSA:")
61 try:
62     A_inv = np.linalg.inv(A)
63     print("Inversa de A:")
64     print(A_inv.round(2))
65 except np.linalg.LinAlgError:
66     print("A no tiene inversa (es singular)")
67
68 # Rango
69 print("\n" + "-" * 30)
70 print("RANGOS:")
71 print(f"rango(A) = {np.linalg.matrix_rank(A)}")
72 print(f"rango(B) = {np.linalg.matrix_rank(B)}")
73
74 # Traza
75 print("\n" + "-" * 30)
76 print("TRAZAS:")
77 print(f"traza(A) = {np.trace(A)}")
78 print(f"traza(B) = {np.trace(B)}")
79
80 # Normas
81 print("\n" + "-" * 30)
82 print("NORMAS:")
83 print(f"Norma Frobenius de A: {np.linalg.norm(A, 'fro'):.2f}")
84 print(f"Norma 1 de A: {np.linalg.norm(A, 1):.2f}")
85 print(f"Norma infinita de A: {np.linalg.norm(A, np.inf):.2f}")
86 print(f"Norma 2 de A: {np.linalg.norm(A, 2):.2f}")
87
88 # Potencia
89 print("\n" + "-" * 30)
90 print("POTENCIA (A^2):")
91 print(np.linalg.matrix_power(A, 2))

```

Listing 19: Ejemplo Completo de Operaciones

15 Referencias

- NumPy Documentation: <https://numpy.org/doc/stable/>
- Linear Algebra with NumPy: <https://numpy.org/doc/stable/reference/routines.linalg.html>
- NumPy User Guide: <https://numpy.org/doc/stable/user/>

Operaciones entre Matrices con NumPy en Python

Documento Técnico

10 de septiembre de 2026

Resumen

Este documento describe las principales operaciones entre matrices que pueden realizarse utilizando la librería `NumPy` del lenguaje Python. Para cada operación se presentan ejemplos prácticos y su correspondiente interpretación matemática.

Índice

1. Introducción	2
2. Creación de matrices	2
2.1. Matrices a partir de listas	2
2.2. Matrices especiales	2
3. Suma y resta de matrices	3
4. Producto por un escalar	3
5. Producto matricial	3
5.1. Producto elemento a elemento (Hadamard)	4
6. Transpuesta	4
7. Determinante	4
8. Inversa	5
9. Resolución de sistemas lineales	5
10. Traza y norma	5
10.1. Traza	5
10.2. Norma	6
11. Autovalores y autovectores	6
12. Rango y otras funciones útiles	6
13. Resumen de funciones	6
14. Conclusión	7

1. Introducción

NumPy (Numerical Python) es la librería fundamental para el cálculo numérico en Python. Proporciona el objeto `ndarray`, una estructura eficiente para almacenar y operar con arreglos multidimensionales, incluyendo matrices.

Una matriz A de tamaño $m \times n$ puede representarse en NumPy mediante un arreglo bidimensional:

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

Para trabajar con matrices se importa la librería de la siguiente manera:

```
1 import numpy as np
```

Listing 1: Importación de NumPy

2. Creación de matrices

Antes de operar con matrices, es necesario crearlas. NumPy ofrece varias funciones para ello.

2.1. Matrices a partir de listas

```
1 import numpy as np
2
3 A = np.array([[1, 2, 3],
4               [4, 5, 6]])
5
6 B = np.array([[7, 8],
7               [9, 10],
8               [11, 12]])
9
10 print("A =\n", A)
11 print("B =\n", B)
```

Listing 2: Creación de matrices con `np.array`

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}, \quad B = \begin{pmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{pmatrix}$$

2.2. Matrices especiales

```
1 Z = np.zeros((3, 3))           # matriz de ceros 3x3
2 U = np.ones((2, 4))           # matriz de unos 2x4
3 I = np.eye(3)                  # matriz identidad 3x3
4 R = np.random.rand(2, 3)      # matriz aleatoria 2x3
```

Listing 3: Matrices de ceros, unos e identidad

3. Suma y resta de matrices

La suma (o resta) de dos matrices $A, B \in \mathbb{R}^{m \times n}$ se define elemento a elemento:

$$(A \pm B)_{ij} = a_{ij} \pm b_{ij}$$

```
1 A = np.array([[1, 2], [3, 4]])
2 B = np.array([[5, 6], [7, 8]])
3
4 S = A + B
5 D = A - B
6
7 print("A + B =\n", S)
8 print("A - B =\n", D)
```

Listing 4: Suma y resta de matrices

$$A + B = \begin{pmatrix} 6 & 8 \\ 10 & 12 \end{pmatrix}, \quad A - B = \begin{pmatrix} -4 & -4 \\ -4 & -4 \end{pmatrix}$$

4. Producto por un escalar

Dado un escalar $\alpha \in \mathbb{R}$ y una matriz A , el producto αA multiplica cada entrada de A por α :

$$(\alpha A)_{ij} = \alpha a_{ij}$$

```
1 A = np.array([[1, 2], [3, 4]])
2 alpha = 3
3
4 print(alpha * A)
5 # [[ 3  6]
6 #   [ 9 12]]
```

Listing 5: Producto por un escalar

5. Producto matricial

Si $A \in \mathbb{R}^{m \times n}$ y $B \in \mathbb{R}^{n \times p}$, el producto $C = AB \in \mathbb{R}^{m \times p}$ se define como:

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

En NumPy se utilizan `np.dot`, el operador `@` o `np.matmul`. **No debe usarse `*`** porque en NumPy realiza el producto elemento a elemento (Hadamard).

```
1 A = np.array([[1, 2, 3],
2               [4, 5, 6]])      # 2x3
3
4 B = np.array([[1, 0],
5               [0, 1],
6               [1, 1]])      # 3x2
7
8 C1 = np.dot(A, B)
9 C2 = A @ B
10 C3 = np.matmul(A, B)
```

```

11
12 print(C1)
13 # [[ 4  5]
14 #  [10 11]]

```

Listing 6: Producto matricial

$$AB = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 4 & 5 \\ 10 & 11 \end{pmatrix}$$

5.1. Producto elemento a elemento (Hadamard)

```

1 A = np.array([[1, 2], [3, 4]])
2 B = np.array([[5, 6], [7, 8]])
3
4 H = A * B
5 print(H)
6 # [[ 5 12]
7 #  [21 32]]

```

Listing 7: Producto Hadamard

6. Transpuesta

La transpuesta de $A \in \mathbb{R}^{m \times n}$, denotada A^T , es la matriz de tamaño $n \times m$ tal que $(A^T)_{ij} = a_{ji}$.

```

1 A = np.array([[1, 2, 3],
2               [4, 5, 6]])
3
4 print(A.T)
5 # [[1 4]
6 #   [2 5]
7 #   [3 6]]

```

Listing 8: Transpuesta de una matriz

$$A^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

7. Determinante

El determinante solo está definido para matrices cuadradas. Se calcula con `np.linalg.det`.

```

1 A = np.array([[4, 7],
2               [2, 6]])
3
4 det = np.linalg.det(A)
5 print(det) # 10.000000000000002

```

Listing 9: Determinante de una matriz

$$\det(A) = \begin{vmatrix} 4 & 7 \\ 2 & 6 \end{vmatrix} = 4 \cdot 6 - 7 \cdot 2 = 10$$

8. Inversa

La inversa A^{-1} de una matriz cuadrada A satisface $AA^{-1} = A^{-1}A = I$. Se obtiene con `np.linalg.inv`.

```
1 A = np.array([[4., 7.],
2               [2., 6.]])
3
4 A_inv = np.linalg.inv(A)
5 print(A_inv)
6 # [[ 0.6 -0.7]
7 #  [-0.2  0.4]]
8
9 # Verificación
10 print(np.allclose(A @ A_inv, np.eye(2))) # True
```

Listing 10: Inversa de una matriz

$$A^{-1} = \frac{1}{10} \begin{pmatrix} 6 & -7 \\ -2 & 4 \end{pmatrix} = \begin{pmatrix} 0,6 & -0,7 \\ -0,2 & 0,4 \end{pmatrix}$$

9. Resolución de sistemas lineales

Para resolver $A\mathbf{x} = \mathbf{b}$ se recomienda `np.linalg.solve`, que es más eficiente y estable numéricamente que calcular $A^{-1}\mathbf{b}$.

```
1 A = np.array([[3., 2.],
2               [1., 4.]])
3 b = np.array([7., 9.])
4
5 x = np.linalg.solve(A, b)
6 print(x) # [1. 2.]
```

Listing 11: Solución de un sistema lineal

$$\begin{cases} 3x_1 + 2x_2 = 7 \\ x_1 + 4x_2 = 9 \end{cases} \implies \mathbf{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

10. Traza y norma

10.1. Traza

La traza es la suma de los elementos de la diagonal principal: $\text{tr}(A) = \sum_i a_{ii}$.

```
1 A = np.array([[1, 2, 3],
2               [4, 5, 6],
3               [7, 8, 9]])
4
5 print(np.trace(A)) # 15
```

Listing 12: Traza de una matriz

10.2. Norma

La norma de Frobenius se calcula con `np.linalg.norm`:

$$\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2}$$

```
1 A = np.array([[1, 2], [3, 4]])
2 print(np.linalg.norm(A))           # 5.477...
3 print(np.linalg.norm(A, 'fro'))    # 5.477...
```

Listing 13: Norma de Frobenius

11. Autovalores y autovectores

Para una matriz cuadrada A , los pares $(\lambda, \mathbf{v})$ que cumplen $A\mathbf{v} = \lambda\mathbf{v}$ se calculan con `np.linalg.eig`.

```
1 A = np.array([[2., 0.],
2               [0., 3.]])
3
4 valores, vectores = np.linalg.eig(A)
5 print("Autovalores:", valores)      # [2. 3.]
6 print("Autovectores:\n", vectores)
```

Listing 14: Autovalores y autovectores

12. Rango y otras funciones útiles

```
1 A = np.array([[1, 2],
2               [2, 4]]) # matriz singular
3
4 print(np.linalg.matrix_rank(A))    # 1
5 print(np.sum(A))                   # 9
6 print(np.sum(A, axis=0))           # [3 6]
7 print(np.sum(A, axis=1))           # [3 6]
8 print(np.linalg.matrix_power(A, 2))
```

Listing 15: Rango, suma y potencias

13. Resumen de funciones

Operación	Función / Operador
Crear matriz	<code>np.array</code> , <code>np.zeros</code> , <code>np.ones</code> , <code>np.eye</code>
Suma / resta	<code>+</code> , <code>-</code>
Producto escalar	<code>*</code> , <code>np.multiply</code>
Producto matricial	<code>@</code> , <code>np.dot</code> , <code>np.matmul</code>
Transpuesta	<code>A.T</code>
Determinante	<code>np.linalg.det</code>
Inversa	<code>np.linalg.inv</code>
Sistema lineal	<code>np.linalg.solve</code>
Traza	<code>np.trace</code>
Norma	<code>np.linalg.norm</code>
Autovalores	<code>np.linalg.eig</code>
Rango	<code>np.linalg.matrix_rank</code>

14. Conclusión

La librería NumPy proporciona un conjunto completo y eficiente de herramientas para realizar operaciones matriciales en Python. Su sintaxis, basada en operadores y funciones vectorizadas, permite escribir código conciso y de alto rendimiento, adecuado tanto para aplicaciones académicas como profesionales en álgebra lineal, ciencia de datos, aprendizaje automático e ingeniería.